

Germline Small-Variant Annotation and Pathogenicity Classification on NCHC (Hands-on)

A VEP-based annotation and point-based in-house ACMG classification workflow for germline small variants

Workflow developed by 李婕瑜 (Chieh-Yu Lee) • 袁自航 (Tzu-Hang Yuan) • Dr. 陳俞安 (Dr. Yu-An Chen)

Advisors: Dr. 陳沛隆 (Dr. Pei-Lung Chen) • Dr. 許書睿 (Dr. Jacob Shu-Jui Hsu) • Dr. 陳倩瑜 (Dr. Chien-Yu Chen)

Computation on NCHC • Draft 2026.

Germline Small-Variant Annotation and Pathogenicity Classification on NCHC (Hands-on)

Germline small variant 致病性分析 · 操作手冊 (Hands-on). 適用需在國網重建此流程的使用者。

本手冊涵蓋三段流程：① VEP 註解 → ② ACMG 致病性分類 → ③ 條件篩選。VEP 的輸出（.vep.tsv）是 ACMG 的輸入，ACMG 的輸出（.vep.ACMG.tsv）是篩選的輸入，請依序執行。

本版重點 ① 新增 submit.sh 支援 sample.list 提交。② filter_acmg.py 位於 ACMG_filter/，同資料夾附 run_filter.sh、gene.list。③ conda 環境安裝位置可自行選擇（/home/\$USER/ 或 /work/\$USER/ 下）。④ VEP 完成後可自動接 ACMG（RUN_ACMG 開關）。⑤ 篩選分「分類模式（OR）／流程模式（AND）」兩種，輸出一律自動含 ACMG_point。

0. 改成自己的：

項目	範例值	換成
帳號 ¹	d03455002	你的登入帳號
計畫代號 #SBATCH -A	MST109178	你 iService 的計畫 ID
通知信 --mail-user	you@example.com	你的 email

¹ 如果你的工作區域是在自己帳號的 /work 下，後面所有的 \$USER 都不需要特別改成自己的帳號。

不要改 所有 /staging/reserve/jacobhsu/reference/VEP/...（參考基因組、VEP cache、plugins、ClinVar/gnomAD 等 custom annotation、ACMG 的 config.json 資料庫路徑）。這是實驗室共用資料，有讀取權限就直接用。先驗證：ls /staging/reserve/jacobhsu/reference/VEP/ 列得出東西即可。

1. 目錄結構（最終長這樣）

```
/work/$USER/VEP-ACMG/
├── VEP-small_variant/          # VEP 註解
│   ├── 00_vep_batch_submitter.py
│   ├── 01_preprocess.sh
│   ├── 02_submit_vep.sh
│   ├── 02_vep.sh
│   └── 03_post_vep.sh
├── ACMG-small_variant/        # ACMG 分類
│   ├── acmg_batch_submitter.sh # 自行跑 ACMG 分類 (Stage 5)
│   ├── sample_acmg.list       # 自行跑 ACMG 分類需設定
│   ├── run_acmg.sh
│   ├── main.py
│   └── modules/
│       ├── acmg_rules.py
│       ├── aggregation.py
│       └── data_loader.py
├── ACMG_filter/
│   ├── rescore.py             # 自訂每個規則的分數以及分類的閾值
│   ├── run_filter.sh          # 設定檔式啟動器 (Stage 6)
│   ├── filter_acmg.py         # 篩選腳本 (Stage 6)
│   └── gene.list              # --gene-list 用的基因清單 (預設空)
├── utils/
│   ├── VEP/
│   ├── ACMG/
│   │   ├── config.json
│   │   └── hg38_BA1exc.tsv
│   ├── job_utils.sh
│   ├── environment.yml
│   └── setup_env.sh           # 互動式安裝 conda 環境 (Stage 2)
├── input_vcf/                 # 你的輸入 VCF
├── sample.list                # VEP 要跑哪些 case (自訂)
├── results/                   # 所有輸出
│   ├── sample/
│   │   ├── VEP_output/
│   │   ├── ACMG_output/
│   │   ├── scripts/
│   │   ├── logs/             # 每個樣本個別執行腳本時所產生的 log
│   │   └── sample_submit.sh
│   └── logs/                  # 送出腳本的 log
├── config.sh                  # 設定 (Stage 3)
└── submit.sh                  # 一鍵送出腳本 (Stage 4)
```

2. Stage 0 • 登入

```
ssh <帳號>@t3-c4.nchc.org.tw # 登入國網
```

3. Stage 1 • Clone github repo

```
cd /work/$USER
git clone https://github.com/Jacob-s-Lab/VEP-ACMG.git
```

4. Stage 2 • 建立環境

2-1 VEP：不用安裝。VEP 使用系統 module 與已裝好的共用 conda 環境（`/opt/ohpc/Taiwania3/pkg/biology/vep/vep_v115`），只要能 load module。

2-2 ACMG：建立 conda 環境 `acmg_rule`。

只需在 第一次 使用時執行。

```
cd /work/$USER/VEP-ACMG/utils
bash setup_env.sh
```

會出現以下互動式選項，可根據使用習慣設定 conda 環境安裝路徑：

Conda Environment Auto-Installer

請選擇您的 Conda 環境安裝路徑：

- 1) /home/user/.conda/envs/ (預設路徑)
- 2) /work/user/.conda/envs/
- 3) 自訂路徑 (手動指定環境與快取路徑)

請輸入選項 (1/2/3):

選項 1 會按照 conda 預設安裝方式將環境及快取建在家目錄下。選項 2 會將環境建在 `/work/$USER/VEP-ACMG/utils/.venv/` 下，快取放在 `/work/$USER/.conda/pkg` 中。

5. Stage 3 • 設定參數

```
cd /work/$USER/VEP-ACMG
```

修改 `config.sh` 頂端自訂區塊。

```
# Fill the mail address if you want to receive email notifications when the job ends or fails.
export USERMAIL=""

# Your working directory (i.e., where this script is located)
export WKDIR="/work/${USER}/VEP-ACMG"

# Automatically run ACMG after VEP completion
## 1 = annotation + pathogenicity classification
## 0 = Only annotation
export RUN_ACMG=1
```

如果 `VEP-ACMG/` 資料夾是建在 `/work/$USER/` 下，不需改動 `WKDIR`。

`# Automatically run ACMG after VEP completion` 見 6.1 說明。

6. Stage 4 • 執行 VEP 註解

流程：把 `*.vcf` 或是 `*.vcf.gz` 放進 `input_vcf/` → 在 `sample.list` 列出要跑的 case → 送出 `submit.sh`。 `submit.sh` 依 `sample.list` 只挑指定 case，自動送出 preprocess → VEP array → merge 三段 job。

如果不會編輯檔案或是要跑所有 `input_vcf/` 中的樣本，可以直接執行 `submit.sh` 不需編輯 `sample.list`，但要確保 `input_vcf/` 中有檔案。

```
# 1) 放輸入 VCF 到 input_vcf/
# 2) 編輯 sample.list (一行一個，# 開頭跳過)
# 3) 送出
cd /work/$USER/VEP-ACMG
bash submit.sh
queue --me
```

sample.list 寫法 每行可為 ① 完整檔名 (`xxx.vcf.gz`) ② 樣本 ID (`xxx`，自動補 `.vcf.gz` / `.vcf`) ③ 短 ID 前綴 (如 `A5034E`，自動對到 `input_vcf/A5034E*.vcf.gz`，排除 `.tbi` 與 `.gvcf.gz`)。用短 ID 時輸出樣本名就用該短 ID。一個前綴對到多個檔會警告，請改寫更完整名稱。

在 `sample.list` 中有問題的寫法／樣本會記錄在 `results/skipped_samples.warnings` 中，沒有出現這個檔案代表寫法沒有問題。

每個樣本完成後，在 `results/<樣本>/VEP_output/` 產生 `<樣本>.vep.tsv` (ACMG 輸入) 與 `<樣本>.vep.mane_plus_clinical.tsv`。

6.1 VEP 完自動接 ACMG

送出 `submit.sh` 後，每個 sample 的 VEP 三段 job 跑完，會用 Slurm 相依性自動送出該 sample 的 ACMG 分類，不需再手動執行 `acmg_batch_submitter.sh`。

如何設定 `RUN_ACMG`：開關就寫在 `config.sh` 頂端的「自訂」區。預設已是 1 (VEP 完自動接 ACMG)；若只想跑 VEP、之後再自己做 Stage 5，改成 0 即可。改好存檔後照常 `bash submit.sh`。

7. Stage 5 • 執行 ACMG 分類 (手動／批次，選用)

已啟用自動串接者可略過本段；本段用於 `RUN_ACMG=0`、補跑或一次重跑全部樣本。

5-1 設定輸入樣本檔案路徑。將所有需要跑 ACMG 分類的檔案「絕對路徑」列在 `sample_acmg.list` 中。

如果不會編輯檔案或是要跑所有 `results/` 中的樣本，可以直接執行 `ACMG-small_variant/acmg_batch_submitter.sh` 不需編輯 `sample_acmg.list`。

在 `sample_acmg.list` 中有問題的寫法／樣本會記錄在 `results/acmg_skipped_samples.warnings` 中，沒有出現這個檔案代表寫法沒有問題。

5-2 手動送出：

```
cd /work/$USER/VEP-ACMG/ACMG-small_variant
bash acmg_batch_submitter.sh
squeue --me
```

結果在 `results/<樣本>/ACMG_output/`：`<樣本>.vep.ACMG.tsv` 與 `<樣本>.vep.mane_plus_clinical.ACMG.tsv`（含 `Pathogenicity_class`、`ACMG_rules`）。

8. Stage 6 • 條件篩選 (filter_acmg.py)

位於 `ACMG_filter/`。分兩種互斥模式（自動判定），輸出一律自動補上 `ACMG_point` 欄位。用 `-h` 看 option（`-h` 免啟用環境即可看）；`--list-columns` 列出全部欄位。

- 分類模式：用到 `--plp` / `--clinVar-plp` / `--dvd-plp` 任一個 → 結果 = 它們的 OR 聯集，且忽略下方所有流程條件。
- 流程模式：完全不用分類旗標時，才依固定順序做 AND 篩選。

8.1 方式 A：設定檔式啟動器 run_filter.sh（建議）

同資料夾內的 `run_filter.sh` 已把環境啟用與路徑處理包好，修改「2) 篩選條件」把要用的條件行前的 `#` 拿掉即可。執行的時候加上要執行的檔案名稱 (e.g., `A5034E.vep.ACMG.tsv`)。

```
cd /work/$USER/VEP-ACMG/ACMG_filter
bash run_filter.sh <檔案名稱> # 也可 sbatch run_filter.sh <檔案名稱> 丟進 Slurm
```

檔案路徑注意 輸入檔案名稱後會尋找 `results/<樣本>/ACMG_output/`，不需給完整路徑。**log 位置** log 一律寫到 `results/logs/`。

8.2 方式 B：直接呼叫 filter_acmg.py

```
cd /work/$USER/VEP-ACMG/ACMG_filter
source ../config.sh
activate_conda_env
python filter_acmg.py -i <input>.vep.ACMG.tsv -o <out>.tsv [options]
```

8.3 選項（一）分類模式（獨立；OR 聯集；忽略流程條件）

Option	作用
<code>--plp</code>	ACMG <code>Pathogenicity_class</code> = Pathogenic / Likely_pathogenic
<code>--clinVar-plp</code>	ClinVar_CLNSIG = Pathogenic / Likely_pathogenic（ <code>--ex-blb</code> 的相對版）
<code>--dvd-plp</code>	DVD 分類 = Pathogenic / Likely_pathogenic

8.4 選項（二）流程模式（AND；依此固定順序）

#	Option	作用
1	<code>--pass-only</code>	只留 FILTER = PASS
2	<code>--min-dp INT</code>	保留 DP ≥ 值（空值濾掉）
3	<code>--acmg-ex RULE[,RULE]</code>	刪除 <code>ACMG_rules</code> 含指定精確 token 者（如 BA1）
—	<code>--ex-blb</code>	去掉 ClinVar 純良性（含 B/LB 且無 conflicting/VUS/P）
4	<code>--min-vaf FLOAT</code>	保留 VAF ≥ 值（空值濾掉）
—	<code>--max-af FLOAT</code>	genome_AF 與 exome_AF 皆 ≤ 值（空值保留）；0.01 / 0.05
5	<code>--gene-list PATH</code>	只留 SYMBOL 在基因清單中的位點（一行一個）
6	<code>--exonic / --noncoding</code>	只留第 1 類 exonic+splice／只留非第 1 類（擇一）
7	<code>--consequence + --mode</code>	自訂 Consequence 關鍵字； <code>--mode include/exclude</code>
8	<code>--acmg-ex-b-rule-only</code>	刪除只有 B 證據、無 P 證據者

#	Option	作用
9	--vushigh	只留 ACMG_point 總分 ≥ 3 (VUS-high)

共用：--list-columns / -v (列欄位 / 顯示每步剩餘列數)。你打 option 的先後不影響結果 (AND 交集與順序無關；只有 -v 中間計數依固定順序)。

8.5 ACMG 點數規則 (一律自動計算 ACMG_point)

P 開頭為正、B 開頭為負；_修飾 (_VERYSTRONG / _STRONG / _MODERATE / _SUPPORTING) 覆蓋等級。--vushigh 只留總分 ≥ 3。

等級	Pathogenic (+)	Benign (-)
very strong	PVS = +8	BA = -8
strong	PS = +4	BS = -4
moderate	PM = +2	BM = -2
supporting	PP = +1	BP = -1

例：PVS1,PM2 = 8+2 = 10；PVS1_STRONG = +4；BP6_STRONG = -4；BA1 = -8。

8.6 範例

```
# 分類模式：留 ACMG-P/LP 或 DVD-P/LP 的聯集 (自動含分數)
python filter_acmg.py -i MDD261-PKD.vep.ACMG.tsv -o class.tsv --plp --dvd-plp -v

# 流程模式：PASS + 去 BA1 + coding + VUS-high (AND)
python filter_acmg.py -i MDD261-PKD.vep.ACMG.tsv -o flow.tsv \
  --pass-only --acmg-ex BA1 --exonic --vushigh -v
```

9. 運算資源 (各 script 的 SBATCH 設定)

Script	分區	cores	記憶體	備註
01_preprocess.sh	ngs13G	2	13 GB	VCF 正規化、切染色體
02_submit_vep.sh	ngs7G	1	7 GB	控制/送 array
02_vep.sh	ngs92G	14	92 GB	VEP 註解 (array)
03_post_vep.sh	ngs7G	1	7 GB	合併、產 TSV
run_acmg.sh	ngs26G	4	26 GB	ACMG 分類 (array)
run_filter.sh	ngs7G	1	7 GB	條件篩選 (選用 sbatch)